

please read the two papers carefully

May 27

??

??

Claude responded: I'll read both papers carefully using the project knowledge search to understand their content.

I'll read both papers carefully using the project knowledge search to understand their content.

Searched project for “eigenstructures unstructured data manifolds”

Searched project for “eigenstructures unstructured data manifolds”

Good, I've got the Belkin & Niyogi Laplacian Eigenmaps paper. Now let me read the CVPR 2026 paper.

Searched project for “CVPR 2026 learning eigenstructures method approach”

Searched project for “CVPR 2026 learning eigenstructures method approach”

The search is only returning the Belkin & Niyogi paper. The CVPR 2026 paper isn't matching my queries. Let me try other search terms specific to its likely content.

Searched project for “neural network learning manifold abstract introduction”

Searched project for “neural network learning manifold abstract introduction”

Now I have the core idea. Let me get more detail on the theory — the optimal approximation theorem, and the loss function.

Searched project for “optimal approximation theorem min-max Courant-Fischer probe functions reconstruction loss”

Searched project for “optimal approximation theorem min-max Courant-Fischer probe functions reconstruction loss”

Excellent. Let me also check the experiments section and related work for completeness.

Searched project for “experiments shape correspondence 3D point cloud surfaces results comparison”

Searched project for “experiments shape correspondence 3D point cloud surfaces results comparison”

I have a thorough understanding of both papers now. Let me give you a careful summary.

I've read both papers carefully. Here's a synthesis tailored to your background — the Velich et al. CVPR 2026 paper is essentially a **modern neural reformulation** of the geometric program that Belkin & Niyogi launched in 2001, so it makes sense to read them together.

Belkin & Niyogi (NIPS 2001) — Laplacian Eigenmaps

The classical foundation. Given points $x_1, \dots, x_k \in \mathbb{R}^l$ sampled from a low-dimensional manifold:

1. **Build a graph** with ϵ -balls or kNN.
2. **Weight edges** with the heat kernel $W_{ij} = e^{-\|x_i - x_j\|^2/t}$ (justified as a discrete approximation to the Laplacian-via-heat-kernel argument: $\mathcal{L}f \approx \frac{1}{t}(f - H_t * f)$ for small t).
3. **Solve the generalized eigenproblem** $Ly = \lambda Dy$ where $L = D - W$, and embed each point x_i as the row vector of the smallest non-zero eigenvectors.

The justification is the variational identity $\frac{1}{2} \sum_{ij} (y_i - y_j)^2 W_{ij} = y^\top Ly$, which is the graph analogue of the Dirichlet energy $\int_{\mathcal{M}} \|\nabla f\|^2 = \int_{\mathcal{M}} \mathcal{L}(f)f$. Minimizing it under orthogonality gives eigenfunctions of the Laplace–Beltrami operator. Locality preservation, natural clustering, and connection to spectral clustering (Shi–Malik) all fall out.

The whole machinery hinges on three choices the user must make: **which operator** (Laplacian), **how to discretize it** (kNN + heat kernel weights), and **how to eigendecompose** the resulting sparse matrix.

Velich, Piven, Bensaïd, Cremers, Dages, Kimmel (CVPR 2026) — Learning Eigenstructures

The thesis is that all three of those choices can be replaced by a single learned forward pass.

****Theoretical pivot.**** Instead of building L and computing $L = V\Lambda V^\top$, they invoke ****optimal-approximation theory**** (Theorem 3.1, min-max / Courant–Fischer; Theorem 3.2, its operator-bounded PCA dual). The eigenvectors of any SPD operator L are ***characterized*** as the orthonormal basis that minimizes the worst-case (or expected) reconstruction error for functions in the class $C_L = \{f: \|f\|_L \leq 1\}$. Importantly, $\lambda_{k+1} = 1/\alpha_k$ where α_k is the min-max error after k basis vectors — so eigenvalues come "for free" from reconstruction quality.

Crucially, if you pick $L = \Delta$ (Laplacian), then $\|f\|_L^2 = \|\nabla f\|^2$ is the Dirichlet energy, and C_L is just the class of smooth, bounded-energy functions. So instead of building Δ , you can sample from this class.

The neural framework. Given a point cloud $\mathcal{P} \in \mathbb{R}^{n \times d}$:

1. A network Φ_θ (transformer for 3D / high-dim) predicts a feature matrix in $\mathbb{R}^{n \times K}$.
2. QR-decompose: $\Phi_\theta(\mathcal{P}) = QR$. Columns of Q are interpreted as the first K eigenvectors of an implicit normalized Laplacian-like operator L_{norm} , ordered $0 = \lambda_1 \leq \dots \leq \lambda_K$.
3. **Mass matrix for free.** Because the first eigenvector of L_{norm} encodes $M^{1/2}\mathbf{1}$, they set $M = \text{diag}(q_1 \odot q_1)$ — the sampling density emerges from the first basis vector, no separate module.
4. **Probe functions.** Generate m random functions by Gaussian-smoothing white noise on the kNN graph (a proxy for sampling uniformly from C_Δ). Project them onto truncated bases Q_k (using M -weighted projection but Euclidean error norm — the asymmetry is what implicitly trains M ; see Appendix B.2).
5. ****Loss**** (Eq. 1): $\mathcal{L}_{\text{rec}} = \frac{1}{mK} \sum_i \sum_k \|f^{(i)} - f_{\text{proj},k}^{(i)}\|_2^2$. They use the PCA-form average (Thm 3.2) rather than min-max (Thm 3.1) for stable optimization.

That's the whole training signal. No operator is ever assembled, no eigensolver is called. At inference, eigenvalues are recovered from the worst-case probe reconstruction error: $\lambda_{k+1} \approx 1/\max_i \|f^{(i)} - f_{\text{proj},k}\|^2$.

A nice degree of freedom. The choice of probe distribution defines which implicit operator you're recovering:

- Gaussian-smoothed noise $\rightarrow$ Laplacian-like basis (the default).
- Curvature-modulated probes $\rightarrow$ Hamiltonian-like $(\Delta + \beta V)$ basis.
- Piecewise-constant on K clusters $\rightarrow$ cluster indicators (when $k = K$) or Walsh–Hadamard-like (when $k < K$).
- Degree-2 polynomials $\rightarrow$ polynomial subspace (a warning: smoothness alone isn't enough; you need *uniform* sampling from the constraint set, which is why naive polynomials don't recover the Laplacian).

You essentially trade "design an operator and discretize it" for "design a probe distribution," which is much easier in high dimensions.

Experiments.

- **1D sanity check** on $[0, 1]$: recovers Fourier harmonics.
- **3D surfaces** (single-shape overfit and generalization across SURREAL/SHREC/FAUST/ABC etc.): eigenfunctions almost identical to cotangent-Laplacian oracle (cosine similarities >0.9 for low frequencies), with reasonable eigenvalue recovery. Generalizes to volumetric point clouds without retraining.
- **High-dim image manifolds**: applied to CLIP/DINOv2 features on Caltech256, CIFAR100, Imagenette, STL-10. Their "Optimal-Approximation Eigenmaps" (a direct generalization of Laplacian Eigenmaps) beats PCA, Isomap, Laplacian Eigenmaps, t-SNE, UMAP on NMI/ARI clustering of the embeddings.

Why these two papers belong together

Belkin–Niyogi gave the *justification* (minimizing $y^T L y \approx$ Dirichlet energy $\rightarrow$ eigenfunctions of Δ). Velich et al. invert the program: instead of *building* the operator to *get* its eigenfunctions, they directly *parameterize* the basis with a network and let the optimal-approximation theorem (a statement equivalent to the Rayleigh-quotient characterization B&N implicitly use) supply the supervision. The graph Laplacian — which B&N showed was the right discrete proxy — becomes one specific choice within a larger family controlled by probe-function design, and they argue it's not always the best one in high dimensions, where graph-Laplacian convergence to the LBO degrades.

The connection back is explicit: their image-manifold method is presented as "a direct generalization of Laplacian Eigenmaps," using the learned spectral basis instead of the graph Laplacian's.

Happy to go deeper on any piece — the proof structure of Theorems 3.1/3.2, the M -norm vs Euclidean-norm asymmetry trick in the loss, the eigenvalue recovery via min-max, or how their "foundation model for spectral bases" idea compares to recent neural-operator work.

???

?

You said: what are the conditions under which the Belkin & Niyogi method should work?

what are the conditions under which the Belkin & Niyogi method should work? sampling density? number of sampled points? etc.

May 27

??

??

Claude responded: Good question.

Good question. The 2001 NIPS paper itself is fairly informal about this — it gives the heat-kernel derivation as a *motivation* rather than a convergence theorem. But the conditions under which it provably works were laid out in follow-up work by Belkin & Niyogi and others (especially Belkin & Niyogi 2005/2008, Coifman & Lafon 2006, Hein et al. 2005, Singer 2006, von Luxburg et al. 2008). Let me lay out what's needed, both from what's stated in the NIPS paper you uploaded and from the broader convergence theory that justifies it.

What the NIPS 2001 paper itself assumes (explicit and implicit)

The heat-kernel derivation in §2.2 reveals the assumptions baked into the method:

$$\mathcal{L}f(x_i) \approx -\frac{1}{t} \left[f(x_i) - (4\pi t)^{-n/2} \int_{\mathcal{M}} e^{-\|x-y\|^2/4t} f(y) dy \right]$$

discretized to

$$\mathcal{L}f(x_i) \approx -\frac{1}{t} \left[f(x_i) - \frac{1}{k} (4\pi t)^{-n/2} \sum_{x_j: 0 < \|x_j - x_i\| < \epsilon} e^{-\|x_i - x_j\|^2/4t} f(x_j) \right]$$

For this discretization to be a valid pointwise approximation of $\mathcal{L}f(x_i)$, *each* of these steps must hold simultaneously:

1. **The Gaussian-on-extrinsic-distance approximates the intrinsic heat kernel.** This needs $\|x - y\|$ small enough that ambient Euclidean distance and geodesic distance agree to leading order — i.e., points must be locally on a near-flat patch of $\mathcal{M}$.
2. ****The Monte Carlo sum approximates the manifold integral.**** This needs the points $\{x_j\}$ to be i.i.d. (or at least uniform) samples from $\mathcal{M}$, and *enough* of them inside the ϵ -ball around x_i .
3. **t (and $\epsilon \sim \sqrt{t}$) is small enough** that the heat-kernel-to-derivative limit is accurate, but **large enough** that the Monte Carlo sum has enough samples inside the ball to be statistically meaningful.

That third point is the crux. It's a **bias–variance tradeoff** that ties t , ϵ , the number of samples k , the intrinsic dimension n , and the curvature of $\mathcal{M}$ together.

The conditions, made explicit

1. Manifold regularity

- $\mathcal{M}$ should be a **smooth, compact, connected Riemannian submanifold** of $\mathbb{R}^l$, ideally without boundary (the NIPS paper sidesteps boundary issues — the LBO with boundary requires boundary conditions, and convergence near $\partial\mathcal{M}$ is notoriously slower; this is why their footnote in the 2026 paper insists on the closed-manifold setting).
- **Bounded curvature** matters: the error in approximating geodesic distance by Euclidean distance is $O(\kappa \|x - y\|^3)$ where κ is sectional curvature, so high-curvature regions need smaller ϵ to remain in the "locally flat" regime.
- ****Bounded reach / no near-self-intersections.**** If the manifold folds back near itself (think Swiss roll with thin gaps), the kNN graph will short-circuit across the gap and the recovered Laplacian corresponds to a **different** manifold than the one you sampled from. The ϵ must be smaller than the local reach $\tau(\mathcal{M})$.

2. Sampling density

- ****Uniform sampling**** is what the derivation literally assumes (the prefactor $\frac{1}{k}$ in the Monte Carlo step is the uniform-measure normalization). With non-uniform density $p(x)$, the graph Laplacian converges to a **weighted** operator: roughly $\mathcal{L}_p f = \Delta f + \frac{\nabla p}{p} \cdot \nabla f$, which is the Fokker–Planck operator, not the LBO. ****This is exactly what Coifman & Lafon (2006) fix with diffusion maps via the α -normalization**** (renormalize W by $D^{-\alpha} W D^{-\alpha}$ before computing L ; $\alpha = 1$ recovers the pure LBO regardless of p). The NIPS paper does **not** do this — so strictly speaking, Belkin–Niyogi recovers the LBO only when sampling is uniform.
- **Density bounded away from zero:** $p(x) \geq p_{\min} > 0$ everywhere on $\mathcal{M}$. Regions sampled too sparsely produce disconnected components in the kNN graph (which the paper explicitly notes as the main failure mode of ϵ -balls).

3. Number of samples and the bandwidth scaling

This is the most quantitative condition, and it's the one practitioners get wrong most often.

For pointwise convergence of the graph Laplacian to $\mathcal{L}$, the bandwidth t (equivalently $\epsilon \sim \sqrt{t}$) must scale with N (number of samples) and n (**intrinsic** dimension) as roughly:

$$t \rightarrow 0, N \cdot t^{n/2+1} \rightarrow \infty$$

(See Hein–Audibert–von Luxburg 2005, Singer 2006, Belkin–Niyogi 2008 for precise rates.)

Intuitively:

- $t \rightarrow 0$ kills the bias (heat-kernel localization).
- $N \cdot t^{n/2}$ is the expected number of points in a ball of radius $\sqrt{t}$ — this needs to grow so the Monte Carlo sum concentrates.

The optimal rate trades these off, giving $t \sim N^{-2/(n+4)}$ and an error rate of $O(N^{-2/(n+4)})$ for pointwise convergence. **This is the curse of dimensionality in disguise:** for $n = 2$ (a surface), $t \sim N^{-1/3}$, manageable. For $n = 10$, $t \sim N^{-1/7}$, you need exponentially more samples to shrink the bandwidth at the same rate. This is one of the empirical reasons Velich et al. argue the graph Laplacian degrades on high-dimensional image manifolds.

For spectral convergence (eigenvalues and eigenvectors), the rates are similar but the constants are worse, and you only get convergence for the *first few* eigenpairs — high-frequency eigenfunctions oscillate faster than the bandwidth can resolve.

4. Choice of ϵ or k

- **ϵ -balls:** must satisfy $\epsilon \rightarrow 0$ but $N\epsilon^n \rightarrow \infty$. Choosing ϵ in practice is the well-known headache the paper itself admits ("difficult to choose ϵ "). Too small $\rightarrow$ disconnected graph; too large $\rightarrow$ loss of locality, geodesics short-circuit.
- ****kNN**:** must satisfy $k \rightarrow \infty$ but $k/N \rightarrow 0$. Easier in practice and what most people use, but kNN graphs converge to a *slightly different* operator than ϵ -ball graphs — the limit involves an additional density factor (von Luxburg et al. 2008). For clustering it usually doesn't matter; for precise Laplace–Beltrami recovery it does.

5. Intrinsic vs ambient dimension

The method's complexity scales with ****intrinsic**** dimension n , not ambient l . This is why it works on n^2 -dim image vectors of a low-dim object manifold. But the curse is in n : image manifolds (e.g., DINOv2 features) often have *effective* intrinsic dimension in the tens, where the bandwidth-vs-samples tradeoff becomes painful. Belkin–Niyogi's own toy examples (bars in 40×40 images, but intrinsic dim 2) are deliberately benign.

6. Noise

The original paper doesn't analyze noise. If samples are $x_i + \eta_i$ with η_i off-manifold noise of scale σ , you need $\epsilon \gg \sigma$ to "wash out" the noise but ϵ small enough for the manifold approximation — another tradeoff. Empirically the method is fairly robust to small noise (their "locality preserving character... makes it relatively insensitive to outliers and noise"), but there's no theorem in the NIPS paper backing this.

A useful mental checklist

The method works well when:

Condition	Why
Manifold is smooth, low-intrinsic-dim, no near self-intersections	Local flatness; kNN doesn't short-circuit
Sampling is roughly uniform	Otherwise you get the Fokker–Planck operator, not the LBO
N is large relative to ϵ^{-n}	Monte Carlo sum has enough samples per ball

Condition	Why
ϵ (or t) is in the sweet spot $\rightarrow 0$ but $N\epsilon^n \rightarrow \infty$	Bias–variance tradeoff
You care about low-frequency eigenpairs	High-freq eigenfunctions need ϵ smaller than their wavelength
Boundary effects negligible (closed manifold or interior points)	LBO with boundary requires explicit BCs

It works poorly or fails when:

- The manifold has high intrinsic dimension (curse of dimensionality in ϵ -vs- N).
- Sampling is highly non-uniform (without diffusion-maps-style α -normalization).
- Points concentrate near a boundary.
- The manifold has thin necks, near self-intersections, or topology you don't want recovered (kNN will glue across them).
- You need fine spectral detail — convergence is guaranteed only for the bottom of the spectrum and only at a rate depending on n .

Connection back to the Velich et al. paper

This list is exactly what the CVPR 2026 paper is responding to. Their critique of the graph Laplacian — "harder to approximate" in high dimensions, "more sensitive to the connectivity of the relationship graph" — is shorthand for the breakdown of conditions 3 and 5 above. By replacing operator discretization with probe-function sampling, they shift the burden: you no longer need $N\epsilon^n \rightarrow \infty$ in the ambient ball; instead you need enough probe functions to characterize the smooth-function class C_Δ , which is a much weaker requirement that doesn't scale with intrinsic dimension the same way. Whether that fully escapes the curse of dimensionality or just relocates it into "how do you sample C_Δ uniformly in high dim" is, I think, a fair open question — and they implicitly acknowledge it when they note the polynomial-probe failure mode (smoothness isn't enough, you need *uniform* sampling of the constraint set).

????

?

You said: does Belkin & Niyogi use radius balls or knn?

does Belkin & Niyogi use radius balls or knn?

May 27

??

??

Claude responded: Both — the paper presents them as two interchangeable variations in Step 1 of the algorithm, and lets the user pick.

Both — the paper presents them as two interchangeable variations in Step 1 of the algorithm, and lets the user pick.

Quoting the algorithm specification directly (§1, Step 1 "Constructing the Graph"):

(a) ϵ -neighborhoods. [parameter $\epsilon \in \mathbb{R}$] Nodes i and j are connected by an edge if $\|x_i - x_j\|^2 < \epsilon$. > **Advantages:** geometrically motivated, the relationship is naturally symmetric.* > **Disadvantages:** often leads to graphs with several connected components, difficult to choose ϵ .*

(b) n nearest neighbors. [parameter $n \in \mathbb{N}$] Nodes i and j are connected by an edge if i is among n nearest neighbors of j or j is among n nearest neighbors of i . > **Advantages:** simpler to choose, tends to lead to connected graphs.* > **Disadvantages:** less geometrically intuitive.*

So both are formally on the table. A few observations worth noting:

The theory in §2.2 is built around ϵ -balls, not kNN. The heat-kernel derivation explicitly truncates the integral to $\{x_j: 0 < \|x_j - x_i\| < \epsilon\}$ and the final weight formula

$$W_{ij} = \begin{cases} e^{-\|x_i - x_j\|^2 / 4t} & \text{if } \|x_i - x_j\| < \epsilon \\ 0 & \text{otherwise} \end{cases}$$

uses the ϵ -ball condition. The justification of " L approximates $\mathcal{L}$ " is stated for the ϵ -ball construction. kNN is offered as a practical alternative without a matching derivation — and as I mentioned in the previous message, kNN graphs converge to a *slightly different* limit operator than ϵ -ball graphs (von Luxburg, Belkin & Bousquet 2008 showed the kNN-Laplacian limit involves an additional density factor). So strictly speaking, the Belkin–Niyogi *theory* is an ϵ -ball theory; kNN is a practical concession.

The kNN variant is symmetrized by OR-ing. Note the "or" in (b): an edge exists if *either* point is in the other's neighbor list. This is the *symmetric kNN* graph (also called *mutual-OR-kNN*), as opposed to the *mutual kNN* graph (AND-ing) or the *directed kNN* graph. The OR construction is chosen to guarantee a symmetric W , which is needed for $L = D - W$ to be a symmetric PSD matrix with real spectrum.

The weighting choice is orthogonal to the connectivity choice. Step 2 offers two weight schemes — heat-kernel ($e^{-\|x_i - x_j\|^2 / t}$) and "simple-minded" (just $W_{ij} = 1$ on edges) — independent of whether (a) or (b) was used for connectivity. So there are effectively four combinations: ϵ /heat, ϵ /binary, kNN/heat, kNN/binary. The heat-kernel weights are what the theoretical derivation justifies; the binary weights are a simplification "which avoids the necessity of choosing t ."

In practice, almost everyone uses kNN with binary or heat weights. Despite the ϵ -ball construction being the theoretically clean one, kNN dominates in practice for exactly the reasons the paper lists: choosing ϵ is brittle (too small disconnects the graph in sparsely sampled regions, too large short-circuits across folds), while k is much more forgiving and adapts automatically to varying sampling density.

???

?

You said: can you describe to me the Belkin & Niyogi method and algorithm in simple terms

can you describe to me the Belkin & Niyogi method and algorithm in simple terms

May 27

??

??

Claude responded: Sure.

Sure. Here's the method stripped of formalism.

The setup

You have a bunch of high-dimensional data points — say, thousands of face images, each one a vector of 10,000 pixel values. You suspect that even though each point lives in 10,000 dimensions, the *actual* variation between faces is governed by only a handful of factors (pose, lighting, expression). Geometrically: your data lies on a low-dimensional curved surface — a *manifold* — sitting inside a much bigger space.

Goal: find a 2D or 3D representation of the data that respects this hidden structure, so that points which are "close on the surface" end up close in your new representation. PCA fails at this because it only looks for flat structure; if your manifold is curved (a Swiss roll is the canonical example), PCA squashes it flat the wrong way.

The core intuition

Imagine your data points are villages, and you want to draw them on a map. You don't know the geography of the country, but you do know which villages are *neighbors* — and roughly how far apart neighboring villages are. Belkin & Niyogi's idea: draw the map so that neighboring villages stay close together, and let the global shape emerge from those local constraints.

That's it. The whole method is "preserve local neighborhoods, and the global structure will take care of itself."

The algorithm in three steps

Step 1: Build a neighborhood graph. Connect each data point to its nearby points. Either "everything within distance ϵ " or " k nearest neighbors" — both work. You now have a graph where edges mean "these two points are close in the original space."

Step 2: Weight the edges by closeness. On each edge, put a weight $W_{ij} = e^{-\|x_i - x_j\|^2/t}$ — a Gaussian that's large for very close points and decays for farther ones. (Or just use weight = 1 for every edge; cruder but parameter-free.) Non-edges get weight 0.

Step 3: Find the embedding that minimizes neighbor distortion. Now ask: if I assign each point x_i a new coordinate y_i in low-dimensional space, how do I make sure neighbors stay close? Penalize this quantity:

$$\sum_{i,j} (y_i - y_j)^2 W_{ij}$$

In words: for every pair of points, look at how far apart their new coordinates are, square it, and multiply by their weight. If two points are heavily connected (W_{ij} big) and you put them far apart in the new space, you pay a big penalty. So minimizing this pushes neighbors together.

The minimization has to be constrained (otherwise the trivial solution is to send all points to the same spot), and the constraint is something like "spread the points out with unit variance."

The magical part

Here's where it becomes elegant. That penalty $\sum_{ij}(y_i - y_j)^2 W_{ij}$ can be rewritten exactly as $y^T L y$, where $L = D - W$ is a matrix called the **graph Laplacian** (D is just a diagonal matrix of row sums of W).

Minimizing $y^T L y$ subject to a normalization constraint is a textbook linear algebra problem: solve the generalized eigenvalue problem $Ly = \lambda D y$ and take the eigenvectors with the smallest (non-zero) eigenvalues. The first eigenvector gives you the first coordinate of every point, the second eigenvector the second coordinate, and so on.

So the whole algorithm reduces to: build a graph, build a matrix from it, compute a few smallest eigenvectors. That's the embedding.

Why this works geometrically

The graph Laplacian L is the discrete cousin of the **Laplace–Beltrami operator** $\mathcal{L}$ on the manifold — the same operator that governs heat diffusion, vibration modes of a drum, and harmonic analysis on curved spaces. Its eigenfunctions are the natural "frequency basis" of the manifold (think Fourier series, but adapted to the shape of the surface rather than a flat interval).

When you take the smallest non-zero eigenvectors, you're picking the *lowest-frequency modes* of the manifold — the smoothest, slowest-varying functions on the surface. These are precisely the functions that change *gradually* as you walk along the manifold, which is exactly what you want for an embedding: nearby points get similar values, distant points get different values.

The heat-kernel weight $e^{-\|x_i - x_j\|^2 / t}$ isn't arbitrary either — it's a discretized approximation of how heat spreads on the manifold, which is why L approximates $\mathcal{L}$ rather than some random operator.

A useful analogy

Think of the graph as a network of springs, where stiffer springs connect closer points. Now imagine the whole structure vibrating. The lowest-frequency vibrational modes of this spring network — the slow, smooth, sweeping motions — *are* the Laplacian eigenvectors. Belkin & Niyogi's embedding is literally: "look at how the manifold likes to wobble at its lowest frequencies, and use those wobble patterns as coordinates."

If your data lies on a circle, the lowest modes are $\sin \theta$ and $\cos \theta$ — and the algorithm rediscovers the angle around the circle. If it lies on a Swiss roll, the lowest modes parameterize the roll along its length and width — unrolling it for you.

What you do with the result

Once you have the embedding, you can:

- **Visualize** high-dim data in 2D/3D (their examples: bar images, Brown corpus words, speech phonemes — all show meaningful clustering).
- **Cluster** — natural clusters in the data appear as clusters in the embedding, which is why this is closely related to spectral clustering (Shi–Malik normalized cuts is essentially the same math).

- **Feed downstream classifiers** with a representation that respects the geometry rather than fighting it.

That's the whole method. The deep idea is that *local* information about who-is-near-whom, propagated through the eigenvectors of the Laplacian, recovers *global* geometric structure — and that this works because the discrete graph Laplacian secretly approximates a continuous geometric operator that's been studied for two centuries.

???

?

You said: so what are the advantages and trade off velich's method compared to this one?

so what are the advantages and trade off velich's method compared to this one?

May 27

??

??

Claude responded: Good comparison to draw — they're solving the same problem with very different machinery, so the tradeoffs are interesting.

Good comparison to draw — they're solving the same problem with very different machinery, so the tradeoffs are interesting.

What Velich et al. gains over Belkin–Niyogi

1. No operator construction, no eigensolver. Belkin–Niyogi has a fixed three-stage pipeline: pick operator (Laplacian) → discretize it (graph Laplacian with kNN + heat-kernel weights) → eigendecompose the resulting matrix. Each stage has known failure modes (wrong operator for the task, bad discretization in high dim, numerically sensitive eigensolves on near-degenerate spectra). Velich collapses all three into one learned forward pass. You hand the network a point cloud, you get the basis out the other side.

****2. Graceful scaling to high intrinsic dimension.**** This is the big one. The graph Laplacian's convergence to the Laplace–Beltrami operator requires the bandwidth-vs-samples scaling $N \cdot \epsilon^n \rightarrow \infty$ with $\epsilon \rightarrow 0$, where n is *intrinsic* dimension. For $n = 2$ (surfaces) this is fine. For $n = 10$ –50 (image manifolds) it's brutal — you'd need exponentially many samples to keep the discretization faithful. Velich's framework shifts the burden away from local-neighborhood density estimation: you sample probe functions from the smooth-function class, and the network learns the right basis directly. Their image-manifold experiments (CLIP/DINOv2 features, 384–512-dim) outperforming Laplacian Eigenmaps on clustering metrics is the empirical payoff.

3. The metric/mass matrix comes for free. In Belkin–Niyogi, the diagonal matrix D in $Ly = \lambda Dy$ acts as a crude proxy for sampling density. If density is non-uniform, you actually recover a Fokker–Planck operator rather than the LBO — and fixing this requires the post-hoc α -normalization of Coifman–Lafon's diffusion maps. Velich bakes this into the architecture: the first eigenvector q_1 of their normalized basis directly encodes $\sqrt{M}$, the sampling density. One forward pass, density and basis jointly estimated, no separate normalization step.

4. Eigenvalues are a byproduct, not a separate computation. The min-max theorem (Theorem 3.1) gives $\lambda_{k+1} = 1/\alpha_k$ where α_k is the worst-case reconstruction error after k basis vectors. So you get eigenvalues from inspecting how well your basis reconstructs probe functions — no additional parameters, no second eigensolve.

5. Operator-agnostic by design. This is subtle but important. Belkin–Niyogi commits to the Laplacian. If you want a different operator — Hamiltonian (Laplacian + potential), biharmonic, anisotropic — you need a different discretization scheme each time, and several of these are notoriously hard to discretize on point clouds. In Velich's framework, the operator is implicitly determined by the *probe distribution*. Want a Hamiltonian basis? Modulate probes by a potential before smoothing. Want a clustering-oriented basis? Use piecewise-constant probes. The hard problem of "how do I discretize this operator on unstructured data" is replaced by the much easier problem of "how do I sample functions from a class with this prior."

6. Generalization across shapes / amortized inference. A Belkin–Niyogi eigendecomposition is per-dataset: every new point cloud requires building a new graph and solving a new eigenproblem. Velich's network, once trained on a large shape collection, can produce spectral bases for unseen shapes via a single forward pass — they explicitly frame this as a path toward "foundation models for spectral bases." For applications that need spectra on many shapes (functional maps, shape correspondence pipelines), this is a meaningful speedup.

****7. Robust to point count at inference.**** Their transformer-backbone setup handles variable-size point clouds at inference (e.g., trained on 4k points, evaluated on 12k — cosine similarities go *up*, not down). Belkin–Niyogi has to be re-run from scratch for each new size, and the eigensolve cost grows as $O(N^3)$ for dense or $O(Nk^2)$ for sparse Lanczos.

What Belkin–Niyogi keeps as advantages

1. No training. Just run it. The biggest practical advantage. Belkin–Niyogi is a closed-form algorithm: graph + eigensolve, done in seconds on a laptop for moderate N . Velich's overfitting setting takes ~8 minutes on an RTX 3090 per shape; the generalization model needs a giant pretraining run on ~200k shapes. For a one-off analysis on a single dataset, the classical method is dramatically cheaper to deploy.

2. Deterministic and reproducible. The output of a sparse eigensolve is reproducible up to sign and degenerate-eigenspace rotation. Velich's output depends on network initialization, the random probe functions drawn during training, optimizer noise, etc. For scientific applications where you need the *same* answer every time (e.g., shape-DNA-style signatures for retrieval), this matters.

3. Mathematically transparent. You can write down exactly what Belkin–Niyogi is computing, prove convergence rates (Belkin–Niyogi 2005/2008, Singer 2006, von Luxburg 2008), and reason about failure modes from first principles. Velich's network learns *an* operator's basis; identifying which operator you got (and proving it's the one you wanted) requires inspecting probe distributions and trusting the optimal-approximation theorem. The theory underneath is sound, but the actual learned object is opaque.

****4. No hyperparameters in the model — only in the graph.**** Belkin–Niyogi has ϵ (or k) and bandwidth t . Velich has ϵ/k and bandwidth σ for the *probe-function generation*, plus network architecture, plus optimizer settings, plus training duration, plus the choice of which probe-family to use. They're honest about this — their Table 2 lists per-shape tuned hyperparameters

for $(k, \text{iterations}, \sigma)$, and they note that recovering exact cotangent-Laplacian eigenvalues requires per-shape tuning. The classical method has a smaller hyperparameter surface.

****5. Sparse, scalable to huge N (for the right N regime).**** For N in the millions on a 2D or 3D manifold, a sparse Lanczos eigensolve on the graph Laplacian is genuinely fast and memory-efficient. Velich's transformer backbone has $O(N^2)$ attention cost, which caps the per-batch point count — they train on 1500–5000 points per shape. So the **scaling regimes** are different: classical wins for huge low-dim point clouds, Velich wins for moderate-size high-dim point clouds.

6. No assumption that the test distribution matches training. Velich's generalization model is only as good as its training corpus. Hand it a manifold genuinely outside the training distribution (a weird CAD topology, an exotic dataset), and the learned prior may mislead. Belkin–Niyogi has no learned prior to mislead — the graph Laplacian is just a function of the data you hand it.

7. Decades of downstream tooling. The graph-Laplacian spectral basis is plugged into everything: spectral clustering, functional maps, diffusion distances, Cheeger inequalities, graph signal processing. Some of these depend on exact algebraic properties (e.g., the discrete Green's identities) that hold for $L = D - W$ by construction but may only approximately hold for a learned basis. Plug-and-play replacement isn't automatic.

When to use which

Use Belkin–Niyogi when:

- Your data is on a low-intrinsic-dim manifold (surfaces, sensor networks, low-dim point clouds).
- N is large but manageable for a sparse eigensolve.
- You want a one-off, reproducible result with no training overhead.
- You need the mathematical guarantees for downstream theoretical work.
- You're doing classical spectral clustering or dimensionality reduction.

Use Velich et al. when:

- Your data lives on a high-dim feature manifold (vision embeddings, language embeddings, scientific feature spaces).
- You need to process many shapes / point clouds and want amortized inference.
- You want a non-Laplacian basis (Hamiltonian, semantic-cluster-aware, etc.) without re-deriving discretization schemes.
- You're building a pipeline where the spectral basis is one component in a larger learned system and end-to-end gradients are useful.
- Sampling density is highly non-uniform and you don't want to deal with α -normalization manually.

The honest summary

Velich et al. is not a strict improvement — it's a different point on the tradeoff surface. They've traded the *generality and transparency* of a closed-form algorithm for the *flexibility and scaling*

behavior of a learned one. For 2D surface meshes processed once, classical methods (cotangent Laplacian with a good mass matrix) remain the gold standard and Velich approximately recovers them. For high-dim image manifolds and amortized inference across shape collections, the learned approach genuinely opens doors that the graph Laplacian struggles with.

The deeper reframing they offer — "stop discretizing operators, sample probe functions instead" — is, I think, the more durable contribution than any specific architecture they propose. That reframing translates the hard problem (operator discretization, which is brittle in high-dim) into a softer problem (probe sampling, which is more forgiving), and that's a strategic shift the field has been groping toward for a while.

????

?

You said: do Belkin–Niyogi talk on how they can generalize to other operators?

do Belkin–Niyogi talk on how they can generalize to other operators?

4:46 PM

??

??

Claude responded: No — and this is actually one of the cleaner points of contrast between the two papers.

No — and this is actually one of the cleaner points of contrast between the two papers.

In the NIPS 2001 paper, Belkin & Niyogi are committed to the **Laplacian / Laplace–Beltrami operator specifically**. The entire justification is built around it:

- The variational argument minimizes the Dirichlet energy $\int_{\mathcal{M}} \|\nabla f\|^2$, which is intrinsically tied to the Laplacian (via the identity $\int_{\mathcal{M}} \|\nabla f\|^2 = \int_{\mathcal{M}} \mathcal{L}(f) f$).
- The weight choice $e^{-\|x_i - x_j\|^2 / 4t}$ is derived from the **heat kernel**, which is the Green's function of the heat equation $\partial_t u = \mathcal{L}u$ — again, the Laplacian and nothing else.
- The whole "approximate the Laplace–Beltrami operator with the graph Laplacian" framing is the conceptual core of the paper.

So there's no discussion of generalizing to other operators — Hamiltonians, biharmonic operators, anisotropic Laplacians, etc. The paper isn't *about* a family of operators; it's about *the* Laplacian and its discrete approximation. Generalizing to other operators would require re-deriving a new discretization scheme from scratch for each one, which is precisely the per-operator difficulty that motivates the Velich approach decades later.

The closest the paper comes to anything operator-adjacent is acknowledging connections to **spectral clustering** (Shi–Malik normalized cuts) and to **spectral graph theory** in general (citing Chung) — but these are still all within the Laplacian family. The normalized-cut connection, for instance, corresponds to the *normalized* graph Laplacian rather than the unnormalized one, so

in a loose sense the paper touches two members of the Laplacian family — but that's a far cry from a general operator framework.

This is exactly the gap Velich et al. fill. Their key reframing is that the operator is determined *implicitly by the probe-function distribution*, not by a hand-derived discretization. So:

- Smooth probes (Gaussian-smoothed noise) → Laplacian-like basis (recovering the Belkin–Niyogi regime).
- Curvature-modulated probes → Hamiltonian $\Delta + \beta V$ basis.
- Piecewise-constant probes → cluster-indicator / Walsh–Hadamard basis.

In other words, Velich et al. provide the operator-generalization that Belkin & Niyogi never attempted — and they do it by sidestepping the very thing that locks Belkin & Niyogi to the Laplacian, namely the need to hand-construct a discrete operator. The 2001 paper's contribution was showing the graph Laplacian *is* a faithful discrete Laplace–Beltrami operator; it had no machinery (and no ambition) to extend beyond that one operator.

One caveat worth stating: it's a little anachronistic to fault Belkin & Niyogi for this. In 2001 the achievement was establishing the Laplacian–heat-kernel–graph correspondence for data representation in the first place — they explicitly note this connection, while known to geometers and spectral graph theorists, had no prior application to data representation. Operator generality was simply not the question on the table; making one operator work rigorously was.

????

?

You said: are there follow up works on Belkin–Niyogi that consider generalization to other operators?

are there follow up works on Belkin–Niyogi that consider generalization to other operators?

4:48 PM

??

??

Claude responded: Yes, several lines of follow-up work extended the graph-Laplacian / Laplace–Beltrami program to other operators, though they're scattered across geometry proce...

Yes, several lines of follow-up work extended the graph-Laplacian / Laplace–Beltrami program to other operators, though they're scattered across geometry processing, diffusion-maps theory, and graph signal processing rather than forming one tidy lineage. Let me give you a search-grounded picture, since this is exactly the kind of literature question where my training-data recall could be stale or incomplete.

Searched the web

Searched the web

There's a rich follow-up literature. Let me get a couple more strands — the anisotropic/Fokker–Planck side (diffusion maps normalization) and the geometry-processing side (Hamiltonian operators on meshes).

Searched the web

Searched the web

I now have a clear map of the follow-up literature. Yes — there are several distinct strands, and they extend Belkin–Niyogi in different directions. Here's the organized picture.

1. The diffusion-maps family — a *parametric family of operators via normalization*

This is the most direct and influential generalization. Coifman & Lafon (2006) and Nadler, Lafon, Coifman & Kevrekidis (2006) showed that by introducing an α -normalization — renormalizing the kernel as $W^{(\alpha)} = D^{-\alpha} W D^{-\alpha}$ before building the graph Laplacian — you get a *continuous family* of limiting operators indexed by a single parameter $\alpha \in [0,1]$. The three landmark cases:

- $\alpha = 0$ corresponds to the classical normalized graph Laplacian; $\alpha = 1$ yields the Laplace–Beltrami operator; and $\alpha = 1/2$ yields the backward Fokker–Planck operator. [PNAS](#)

The key conceptual point is that while the graph Laplacian based on a kernel with $\alpha = 1$ captures the geometry of the data, with the density playing no role, the other normalizations take into account also the density of the points on the manifold. `:antCitation[] {citations="d1b85a1d-166e-4d37-ad6e-8fbd9b49c46f" injected="space"}` So this single knob lets you decouple geometry from sampling density — and lands you on different operators (pure geometric Laplacian vs. density-aware Fokker–Planck). Belkin–Niyogi corresponds essentially to the $\alpha = 0$ (or unnormalized) corner of this family, which silently entangles geometry and density. The diffusion-maps line is the cleanest answer to "can the same machinery yield other operators": *yes, just by changing the normalization exponent.*

Berry & Sauer pushed this further to a much larger class: they extended this approach to a much larger class of anisotropic local kernels and showed a connection between the moment expansion of the kernel and the differential operators that can be approximated, allowing the approximation of the forward and backward Fokker–Planck operators associated with a large class of Itô diffusions, provided a kernel with prescribed first and second moments is used. This is striking because it's a precursor in spirit to Velich's "choose the operator implicitly" idea — except here the operator is encoded by the kernel's *moments* rather than by a learned probe distribution. [arxiv](#)

2. Vector / higher-order operators — beyond scalar functions

The second strand generalizes from operators acting on *functions* (0-forms) to operators acting on *vector fields* and *differential forms*:

Connection Laplacian (Vector Diffusion Maps). Singer & Wu introduced VDM, which is a mathematical and algorithmic generalization of diffusion maps and other non-linear dimensionality reduction methods such as LLE, ISOMAP and Laplacian eigenmaps. While existing methods are either directly or indirectly related to the heat kernel for functions over the data, VDM is based on the heat kernel for vector fields. They proved the relation between VDM and the connection Laplacian operator for vector fields over the manifold. A later paper extended this even further: they present a unified framework for approximating other

Connection Laplacians over the manifold by considering its principal bundle structure — so it's explicitly a *framework for a family* of connection Laplacians, not just one. [arxiv + 2](#)

Hodge / higher-order graph Laplacians. Another line estimates the Hodge Laplacians on graphs as approximations to the Laplace–Beltrami operator *on differential forms* (not just functions), with error bounds for the associated Dirichlet forms — connecting to Betti numbers and topological data analysis.

Helmholtzian (1-Laplacian). Chen, Meilă & Kevrekidis proposed the "Helmholtzian Eigenmap," where the manifold Helmholtzian (1-Laplacian) operator Δ_1 generalizes the Laplace–Beltrami operator to vector fields on a manifold. They note while higher order Laplacians have been introduced and studied, this work is the first to present a graph Helmholtzian constructed from a simplicial complex as an estimator for the continuous operator in a non-parametric setting, useful for analyzing flows and vector fields via the Helmholtz–Hodge decomposition. [arxiv](#)[arxiv](#)

3. Task-driven / data-driven kernels

Coifman & Lafon also gestured at operators shaped by an auxiliary function: in some cases one is interested in constructing diffusion kernels that are data- or task-driven; as an example, given an empirical function $F(x)$ on the data, one would like to find a coordinate system in which the first coordinate has the same level lines as F . This is conceptually the closest classical analogue to Velich's "design the probe distribution to encode a prior" idea — bending the operator toward a task rather than taking the bare geometric Laplacian. [PNAS](#)

How this contextualizes Velich et al.

So the answer to your question is: **yes, extensively** — but the classical follow-ups generalize the operator by changing the *construction recipe* (normalization exponent, kernel moments, the cochain degree / bundle you build the graph operator on). Each new operator still requires someone to design and analyze a new discretization scheme: a new normalization, a new simplicial-complex construction, a new local-PCA-and-alignment step for the connection Laplacian, and so on. The convergence theory has to be redone each time.

What's genuinely new in Velich et al. is the *mechanism* of generalization. Instead of hand-deriving a discrete operator for each target, they fix one learning machine and change only the **probe-function distribution**, letting the optimal-approximation theorem map "distribution of constrained functions" $\rightarrow$ "implicit operator whose eigenbasis optimally reconstructs them." Their Hamiltonian-via-curvature-modulated-probes and cluster-indicator-via-piecewise-constant-probes experiments are the same *spirit* as the diffusion-maps family (one framework, many operators) but operationalized through learning and probe design rather than through normalization algebra and moment matching.

A fair way to position it: the diffusion-maps / connection-Laplacian / Hodge lineage already established that the graph-Laplacian program *can* be generalized to a whole zoo of operators. Velich's contribution isn't claiming that's impossible classically — it's arguing that the classical route (per-operator discretization, which gets brutal in high dimension and for exotic operators) can be replaced by a single learned, discretization-free, probe-driven pipeline. Whether that fully delivers on operators where the classical discretizations are already well-understood (like the cotangent Laplacian on meshes, where they only *approximately* match the oracle) versus where they shine — high-dimensional or unstructured settings — is the live empirical question.

If you want, I can pull the specific convergence-rate results for any of these (the connection Laplacian rate in Singer–Wu, or the Hodge-Laplacian Dirichlet-form bound) so you can compare the sample-complexity story against the curse-of-dimensionality discussion we had earlier.

????

?

You said: please make me an easy to understand intuitive explanation for the difference between velich's work the the latests works that follows Belkin–Niyogi.

please make me an easy to understand intuitive explanation for the difference between velich's work the the latests works that follows Belkin–Niyogi. make it ismple to understand. please look up the web to be accurate

4:54 PM

??

??

Claude responded: Let me make sure I'm accurate on the latest follow-up works before drawing the contrast.

Let me make sure I'm accurate on the latest follow-up works before drawing the contrast. Let me do a couple of focused searches.

Searched the web

Searched the web

This is very useful — there's a recent neural method (NEO) that's the perfect foil for explaining what makes Velich's approach different. Let me check one more thing about the classical (non-neural) follow-up line to round it out.

Searched the web

Searched the web

Now I have everything I need. There are two very recent neural follow-ups (Neural Laplacian Operator, ToG 2024; NEO, SIGGRAPH 2026) that make a *much* sharper contrast with Velich than the classical diffusion-maps line — and crucially, they reveal what's actually distinctive about Velich. Here's the intuitive explanation.

The big picture

Belkin & Niyogi (2001) started a 25-year quest: *given scattered data points, find the "natural vibration patterns" (eigenfunctions) of the shape they lie on*. Everyone agrees these vibration patterns are incredibly useful. The whole story since then is about **how you get them** when all you have is a messy cloud of points.

There are two families of recent follow-ups, and Velich sits in a genuinely different spot from both.

Family A: "Build a better operator, then crack it open"

This is the classical lineage and most of the modern neural work too. The recipe is always:

1. Build some operator (a matrix) from the points.
2. Eigendecompose it to get the vibration patterns.

The follow-ups just keep improving **step 1** — making the matrix more faithful to the true shape.

- ****Diffusion maps**** (Coifman–Lafon) added a tuning knob (the α exponent) so the matrix can represent a *whole family* of operators — pure-geometry Laplacian, or density-aware Fokker–Planck — depending on how you normalize. $\alpha = 0$ gives the normalized graph Laplacian, $\alpha = 1$ the Laplace–Beltrami operator, $\alpha = 1/2$ the backward Fokker–Planck operator. [antCitation\[\]](#){citations="e31f5ef3-8d21-451b-94f8-5a217c08036e" injected="space"}
- **Vector diffusion maps / Hodge / Helmholtzian** extended the matrix to act on *vector fields* instead of just functions, reaching operators like the connection Laplacian.
- **The newest neural versions** keep the same two-step recipe but use a network to build the matrix or amortize the eigensolve:
 - **Neural Laplacian Operator** (ToG 2024) trains a graph neural network to predict the *entries* of the Laplacian matrix on a kNN graph. Instead of manually setting edge weights, the network learns them, an extra MLP estimates the mass matrix, and it's trained by imitating the ground-truth Laplacian's behavior on probe functions. [The Moonlight](#)
 - **NEO** (SIGGRAPH 2026) trains a network to *predict the eigenfunctions directly* in one fast forward pass, then cleans them up with a tiny classical eigensolve. A mass-aware neural operator predicts redundant basis functions, which are M-orthonormalized, then the discrete Laplacian is projected into the low-dimensional subspace and a small dense eigenproblem yields the final LBO eigenfunctions via Rayleigh–Ritz refinement. [GitHub](#)

The crucial thing all of Family A shares: **there's a "right answer" they're chasing**. They all target *the Laplacian* (or a known cousin of it). NEO and Neural Laplacian Operator are even trained with *supervision* — they compare against the ground-truth cotangent Laplacian computed from the original mesh and try to match it. The ground-truth Laplacian operator is defined on a manifold mesh, and they train the network so the learned operator behaves similarly to the ground-truth one. [arXiv](#)

Family B: Velich — "Skip the operator entirely; describe what you want the basis to do"

Velich flips the logic. They never build an operator at all, and there's no ground-truth operator to imitate. Instead:

You hand the network examples of the kind of functions you care about ("probe functions"), and ask it: find me the most efficient basis for representing functions like these.

That's the entire idea. The deep theorem (optimal-approximation theory) guarantees that the most efficient basis for a class of functions *is* the eigenbasis of an operator — so by chasing "efficient representation," you get eigenfunctions for free, without ever writing down the operator.

Here's the intuition with an analogy:

Family A is like reverse-engineering a piano. You want to know a piano's natural notes (its vibration modes). Family A says: carefully measure the piano's strings, build an accurate physical model of it (the operator), then compute its resonant frequencies. Better measurements → better model → better notes. The newest neural versions just measure and compute faster, but they're still building the piano model and comparing to a "true" piano.

Family B (Velich) is like listening to what songs people play and inferring the notes. You never model the piano. You just watch which melodies ("probe functions") tend to occur, and ask "what's the smallest set of building-block tones that can reconstruct all these melodies efficiently?" Those building blocks turn out to *be* the piano's natural notes — but you got them by studying the music, not the instrument.

The two differences that actually matter

1. Supervised vs. unsupervised — is there a "correct answer"?

The new neural Family-A methods (NEO, Neural Laplacian Operator) need the **ground-truth Laplacian** (from meshes) to learn from. They're imitators: their goal is to reproduce what the classical cotangent Laplacian would have given, just faster or more robustly on point clouds.

Velich needs **no ground truth at all**. It's trained purely by "reconstruct these probe functions well." It's not trying to copy any existing operator — it discovers a basis from scratch. This is why it can run on data where no ground-truth operator even exists (high-dimensional image-feature manifolds), where Family A's mesh-based supervision simply isn't available.

2. One fixed operator vs. a dial you control

Family A is locked to the Laplacian (that's the answer they're imitating or approximating). To get a *different* operator, someone has to derive a new construction — a new normalization (diffusion maps), a new simplicial complex (Hodge/Helmholtzian), a new supervised target.

In Velich, the operator is chosen implicitly by **which probe functions you feed in** — and changing it is trivial:

- Smooth random functions → Laplacian-like basis.
- Functions bumped up near high-curvature regions → Hamiltonian (Schrödinger-like) basis.
- Functions that are constant within clusters → cluster-detector basis.

You change the operator by changing the *examples*, not by re-deriving math. That's the conceptual freedom — you've moved the hard design choice from "how do I discretize this operator?" (genuinely hard, especially in high dimensions) to "what do example functions look like?" (much easier to specify).

A subtle but important nuance

Notice that **probe functions show up in both NEO/Neural-Laplacian-Operator AND Velich** — but they play opposite roles, and this is the clearest way to see the difference:

- In **Neural Laplacian Operator** (Family A), probe functions are a *measuring stick to imitate a known answer*: "make my learned operator act on these probes the same way the true Laplacian does." The probes test fidelity to a target.

- In **Velich** (Family B), probe functions are the *definition of the question itself*: "the operator I want is whatever operator makes these probes maximally compressible." The probes *create* the target.

Same tool, completely inverted philosophy. Family A uses probes to *check* against ground truth; Velich uses probes to *replace* the need for ground truth.

One-sentence summary

The latest Belkin–Niyogi descendants (diffusion maps, Hodge/connection Laplacians, and the new neural NEO / Neural Laplacian Operator) are all still **building or imitating a known operator and extracting its eigenfunctions** — getting faster, more robust, or covering more operator types, but always chasing a "correct" Laplacian-shaped answer. Velich **throws away the operator-building step entirely**, and instead learns the eigenbasis directly by asking for the most efficient way to represent a chosen family of example functions — turning "pick and discretize an operator" into the much easier "pick what your example functions look like," with no ground-truth supervision needed.

If it'd help, I can turn this into a single side-by-side diagram (the piano analogy mapped onto the actual pipelines), or go deeper on any one of the modern methods — NEO in particular is the most direct neural competitor and worth a closer read alongside Velich.

????

?

You said: so what are the "weaknesses" of methods like diffusion maps and belkin and nyiogi compared to me?

so what are the "weaknesses" of methods like diffusion maps and belkin and nyiogi compared to me?

5:02 PM

??

??

Claude responded: I think there's a small slip in the phrasing — "compared to me.

I think there's a small slip in the phrasing — "compared to me." I'll assume you mean **compared to Velich et al.** (the method in your project), not to you personally. If you actually meant something else, tell me.

That said — given you're reading these papers in a project built around the Velich work, and the corresponding author is at the Technion (Kimmel's group), I should ask directly rather than assume: **are you one of the authors, or otherwise personally involved in this work?** It changes how I'd frame things (e.g., whether to be more adversarial and stress-test the claims for a rebuttal/defense, versus explain them neutrally). I'll answer the substance either way below, framed as Velich vs. the classical methods.

Here are the weaknesses of Belkin–Niyogi and diffusion maps **relative to the Velich approach** — i.e., the specific pain points Velich is designed to avoid. I'll be honest about which are real

wins and which are more arguable, since for any serious use (especially if this is your own paper) the arguable ones are exactly where reviewers will push.

1. They force you to discretize an operator — and that's brittle in high dimensions

Belkin–Niyogi and diffusion maps both *build a matrix* (the graph Laplacian) as a stand-in for the continuous operator. That approximation is only faithful when the bandwidth-vs-samples scaling holds: roughly $N \cdot \epsilon^n \rightarrow \infty$ with $\epsilon \rightarrow 0$, where n is **intrinsic** dimension. For surfaces ($n = 2$) this is easy. For image-feature manifolds (n in the tens) it's brutal — you'd need exponentially more samples to keep the discretization honest. Velich never discretizes an operator, so it sidesteps this particular failure mode. **This is the strongest and most defensible advantage.**

2. Non-uniform sampling silently changes the operator you recover

This is the cleanest concrete weakness. Belkin–Niyogi's unnormalized graph Laplacian, when sampling density $p(x)$ is non-uniform, doesn't converge to the Laplace–Beltrami operator at all — it converges to a density-distorted operator (roughly $\Delta f + \frac{\nabla p}{p} \cdot \nabla f$). You think you're measuring geometry; you're actually measuring geometry tangled with how you happened to sample.

Diffusion maps **fixes exactly this** with the $\alpha = 1$ normalization (that was its whole selling point — separating geometry from density). So this weakness applies ***much more*** to Belkin–Niyogi than to diffusion maps. Velich's angle is different: the density (mass matrix) is recovered automatically as a byproduct (the first basis vector encodes it), so you don't have to know or choose the right normalization in advance.

Be careful here in any writeup: it's not fair to lump diffusion maps in with Belkin–Niyogi on density-handling. Diffusion maps already solved it. Velich's claim has to be "we get it *for free and unsupervised*," not "they can't do it."

3. One operator, fixed. Changing it means re-deriving math.

Both classical methods are tied to the Laplacian family. Want a Hamiltonian ($\Delta + V$), a biharmonic, an anisotropic operator? Each one needs a new hand-derived discretization (or, for diffusion maps, a new normalization/kernel-moment scheme à la Berry–Sauer). In Velich, you change the operator by changing the probe distribution — no new discretization. **This is a real flexibility win, though note diffusion-maps-with-anisotropic-kernels already reaches a fairly broad operator family, so the gap is "easier/unified" rather than "possible vs. impossible."**

4. Per-dataset cost, no amortization

A Belkin–Niyogi or diffusion-maps eigendecomposition is computed from scratch for every new dataset: build graph, run eigensolver, done — every single time. There's no notion of "train once, reuse." Velich's network, once trained, produces bases for unseen shapes in a forward pass (their "foundation model for spectral bases" framing). If you process many shapes, this amortizes; for a single one-off analysis, it doesn't help (and Velich is actually *more* expensive there — see the honest caveats below).

5. Hyperparameters that break the graph, not just degrade it

Choosing ϵ (or k) in the classical methods has a cliff: too small $\rightarrow$ disconnected graph (the method fails outright); too large $\rightarrow$ geodesics short-circuit across folds (you recover the wrong

manifold). The failure is discontinuous. Velich's probe-generation hyperparameters are smoother in their effect — they found the **basis** is fairly insensitive to them (only exact **eigenvalue** recovery needs tuning).

6. Sensitivity to the chosen distance metric

Both classical methods take the ambient distance (or cosine similarity) at face value when building the graph. On high-dim feature manifolds, raw distances are noisy and the kNN graph inherits that noise directly into the operator. Velich's learned extractor can, in principle, learn a representation that's more forgiving — though this is more of a "plausible advantage" than a cleanly demonstrated one.

Now the honest caveats — where the classical methods are *not* weaker (and reviewers will say so)

Since you're clearly engaging deeply, you should hold the advantages above against these:

- **No training, fully deterministic.** Belkin–Niyogi/diffusion maps give the same answer every run in seconds. Velich needs training, has init/probe randomness, and costs ~8 min/shape in the overfitting setting. For a single shape, the classical methods are *strictly* cheaper and more reproducible.
- **Provable convergence.** The classical methods have rigorous consistency theorems (Belkin–Niyogi 2008, Coifman–Lafon, Singer). Velich relies on the optimal-approximation theorem plus an empirical learning procedure; *which* operator you actually recovered is harder to certify.
- **Scaling to huge *Non* low-dim manifolds.** Sparse Lanczos on a graph Laplacian handles millions of points on a surface cheaply. Velich's transformer has $O(N^2)$ attention and trains on 1500–5000 points; the classical methods win decisively in that regime.
- **The hard problem is relocated, not eliminated.** "Pick and discretize an operator" becomes "pick a probe distribution that uniformly samples the right function class." Velich's own polynomial-probe experiment shows this isn't trivial — smoothness alone isn't enough; you need *uniform* sampling of the constraint set. So the difficulty is moved, and arguably softened, but not made free.
- **On meshes, they only approximately match the oracle.** Where the classical cotangent Laplacian is the gold standard (clean 2D surface meshes), Velich *approximates* it (cosine sims drop at high frequencies) rather than beating it. The win is in the regimes where the classical discretizations struggle, not everywhere.

So the fair framing: Velich's advantages are sharpest on **high-dimensional, non-uniformly-sampled, unstructured data where you may want a non-Laplacian operator and you process many instances**. On **clean low-dimensional one-off problems**, the classical methods remain cheaper, more reproducible, and better-guaranteed — and diffusion maps in particular already neutralizes the density-handling weakness that hits Belkin–Niyogi hardest.

Want me to turn this into a tight comparison table (axis = property, columns = Belkin–Niyogi / diffusion maps / Velich) so you can see exactly where each one wins and loses? That's often the most useful form if this is feeding into your own work.

????

?

You said: can you explain what does this means - "or, for diffusion maps, a new normalization/kernel-moment scheme à la Berry–Sauer".

can you explain what does this means - "or, for diffusion maps, a new normalization/kernel-moment scheme à la Berry–Sauer". in simple terms. how it relates to choosing alpha?

5:24 PM

??

??

Claude responded: Good question — this connects two things I mentioned separately, so let me untangle them.

Good question — this connects two things I mentioned separately, so let me untangle them. The short version: choosing α is the *baby version* of the idea, and Berry–Sauer is the *grown-up, general version* of the same trick.

Start with what α actually does

In diffusion maps you build a kernel matrix from your points — basically a Gaussian on distances, $W_{ij} = e^{-\|x_i - x_j\|^2 / \epsilon}$. Then before turning it into a graph Laplacian, you **renormalize** it:

$$W_{ij}^{(\alpha)} = \frac{W_{ij}}{(d_i d_j)^\alpha}, \text{ where } d_i = \sum_j W_{ij}$$

Here d_i is roughly "how densely sampled the neighborhood around point i is." Dividing by it (raised to the power α) is a way of **discounting the effect of sampling density**.

The magic is that the single number α controls *which continuous operator* your matrix converges to:

- $\alpha = 0$: normalized graph Laplacian (density and geometry tangled together).
- $\alpha = 1/2$: backward Fokker–Planck operator.
- $\alpha = 1$: pure Laplace–Beltrami operator (density fully cancelled — pure geometry).

So **one knob** → **a small menu of three operators**. That's the diffusion-maps contribution: a dial that picks among a handful of related operators, all in the Laplacian family.

The limitation of the α knob

But notice: α only slides you along *one specific axis*. All three operators it reaches are close cousins — Laplacian-with-different-density-weighting. You **cannot** reach, say, an *anisotropic* operator (one that diffuses faster in some directions than others), or one with a

potential added (a Hamiltonian, $\Delta + V$), just by turning α . The knob simply doesn't have those settings. To get those, classical theory said you'd need to hand-derive a whole new construction.

What Berry–Sauer generalized

Berry & Sauer asked: *what is it about the kernel that decides which operator you converge to?* And the answer they found is — **the local shape (the "moments") of the kernel.**

Here's the intuition. Forget the Gaussian for a second. A kernel just says "how much does point i talk to its neighbors, and in what pattern?" You can describe that talking-pattern by two simple statistics in each little neighborhood:

- **First moment** — is the kernel *lopsided*? Does it lean toward one direction (talk more to neighbors on the left than the right)? A lopsided kernel injects a *drift* term — a first-derivative term — into the limiting operator.
- **Second moment** — is the kernel's spread *the same in all directions* (a round blob), or *stretched* (an ellipse)? A round blob gives you the plain isotropic Laplacian; a stretched one gives you an *anisotropic* Laplacian that diffuses preferentially along certain directions.

Their result (loosely): **if you want to converge to a particular differential operator, design a kernel whose first and second moments match the drift and diffusion terms of that operator.** Match the moments $\rightarrow$ you get the operator you wanted, in the large-sample limit.

So instead of one knob (α) sliding along one axis, you now have a **recipe for building a custom kernel to hit a much larger class of operators** — in particular the whole family of Fokker–Planck operators for Itô diffusions (think: any drift + any anisotropic diffusion).

How they relate — the clean analogy

Choosing α is like a radio with a single tuning dial that only spans three preset stations (all in the same genre).

****Berry–Sauer's moment-matching**** is like being handed the circuit design and told: "shape the kernel's moments however you like, and you can tune to **any** station in a whole frequency band." The α knob turns out to be one special, restricted way of adjusting the kernel — so it's a **special case** of the more general moment-shaping idea.

Concretely: tweaking α is one particular way of altering the kernel's effective density-weighting, which changes one piece of its moment structure. Berry–Sauer says "that's just one lever; here are all the levers, and here's the rule connecting lever positions to the resulting operator."

Why I mentioned it in the "weakness" context

The point I was making about Velich was: *to reach a non-Laplacian operator, the classical world makes you do design work up front.*

- With diffusion maps, that design work is at minimum "pick the right α ," and for anything beyond the Laplacian cousins, "hand-design a kernel with the right moments" (Berry–Sauer). Either way, **you reason about the operator and engineer the kernel to match it.**

- Velich's pitch is that you skip all of that: you don't engineer a kernel to have prescribed moments, you just *show the network example functions* of the type you care about, and the matching operator emerges implicitly.

So Berry–Sauer is actually the *closest classical ancestor* to Velich's philosophy — both are saying "the operator is determined by something you control upstream" (kernel moments for Berry–Sauer, probe-function distribution for Velich). The difference is that Berry–Sauer still requires you to **analytically design** that upstream object to match a known target operator, whereas Velich **learns** the basis from examples without ever writing the operator (or its moments) down.

A useful way to file it: α = one dial, three stations. Berry–Sauer = moment-matching, a whole band of stations, but you must hand-build the antenna. Velich = don't build an antenna at all; just play it the music you want and let it infer the notes.